
DEVELOPMENT PLANNING DOCUMENT

for

Encost Smart Graph Project

Version 1.0

Prepared by: Alexander Stokes
SoftFlux Engineer

SoftFlux

May 20, 2023

Contents

1	Introduction/Purpose	4
1.1	Purpose	4
1.2	Document Conventions	4
1.3	Intended Audience and Reading Suggestions	5
1.4	Project Scope	5
2	Specialized Requirements Specification	5
2.1	Credentials	5
2.2	Login Failure	5
2.3	User input accessibility	6
2.4	Case sensitivity of username	6
2.5	Invalid device information	6
2.6	Username and passwords	6
3	Product Backlog	7
4	Sprint Details	9
4.1	Sprint #1 <May 10 - May 13>	9
4.1.1	Product Backlog Items	9
4.1.2	Sprint Tasks	9
4.1.3	Software Design	10
4.1.4	Account Login logic	10
4.1.5	isVerified boolean	10
4.1.6	User input accessibility	10
4.1.7	UserVerifier static	11
4.1.8	Read Password character masking	11
4.1.9	Feature options output prompt	11
4.1.10	Feature identification	11
4.1.11	Software Testing	12
4.1.12	Sprint Task Completion	12
4.2	Sprint #2 <May 14 - May 17>	14
4.2.1	Product Backlog Items	14
4.2.2	Sprint Tasks	14
4.2.3	Software Design	15
4.2.4	Software Testing	17
4.2.5	Sprint Task Completion	17
4.3	Sprint #3 <May 18 - May 21>	19
4.3.1	Product Backlog Items	19
4.3.2	Sprint Tasks	19
4.3.3	Software Design	19
4.3.4	Software Testing	21

4.3.5	Sprint Task Completion	21
5	Conclusion	22

Revision History

Name	Date	Reason for Changes	Version

1 Introduction/Purpose

1.1 Purpose

This document is Software Design Specification for the software Encost Smart Graph Project (ESGP) for the company Encost, which has the purpose of optimising the use and connectivity between devices. This software allows the user to view the devices and their connections, provides graph visualisation of Encost's devices, as well as providing summary statistics about each device. The purpose of this Software Development planning is to inform how the software's implementation was developed over a series of sprints, using Agile SCRUM.

1.2 Document Conventions

- ESGP - Encost Smart Graph Project
- SRS - Software Requirements Specification
- SDS - Software Design Specification
- STP - Software Test Plan

1.3 Intended Audience and Reading Suggestions

The project manager and developers are the targeted audiences. This document will inform how the system was developed, and how the modules were segmented and sequentially developed. This document is for developers and maintainers to be informed how the system was developed, and what decisions were made when implementing it, either for future extensions or fixing issues.

1.4 Project Scope

Using STP #4. The implementation will follow the accepted Software Test Plan #4, which uses Software Design Specification #3. Softflux is only responsible for the base version of the software. This test plan only accounts for an English version, with the following functionalities:

- User login and categorisation
- User input and output
- Graph visualising
- Data processing
- File parsing

Please note: Data summary statistics will not be included in this version

2 Specialized Requirements Specification

2.1 Credentials

The test document incorrectly specifies that there is no mentioned way to read the user credentials. However, this implementation will follow the test document's method, such that the credentials will be loaded into a hashmap and will be hardcoded.

2.2 Login Failure

The STP has specified a login failure prompt to include two options. 1. Try again, 2. Go Back. The accepted SDS specifies that instead, this option should allow the user to 'Continue as Community User', because this particular change was not directly addressed by the STP this implementation will follow the SDS logic.

2.3 User input accessibility

A modification has been used to add a string ':' before a user's input, to make it clear to the user the program is expecting input. The STP has used this in its UI specifications, however, does not have it on the same line as the input. It will be held in a constant variable to allow easy modifiability if in future the developer wishes to modify this string.

2.4 Case sensitivity of username

It will be assumed that usernames are not case sensitive

2.5 Invalid device information

It will be assumed if a single device is invalid when parsing a dataset, the entire dataset is rejected.

2.6 Username and passwords

It was noted in the provided usernames/passwords that one username was spelt 'encors-tUserH', this was assumed to be a misspelling and was corrected to 'encostUserH'.

3 Product Backlog

- Categorising Users

Emma Greenfield is an employee at Encost. She wants to access the extra feature of the system and knows login details. She needs to be able to select to log in.

Olivia Hartman is a community user, she just wants to use the system without wondering whether she has a login or not. She needs to be able to select to be a community user, without having to think too hard.

- ESGP Account Login

Mia Carmichael wasn't sure if she has the right credentials. She just wants to try if she has the right credentials, but otherwise accesses the system with restricted features. She needs the system to allow her to continue as a community user if she fails.

Benjamin Mercer is a community user with a very old keyboard. He often mistypes keys, which leads him to choose the wrong thing. He needs to be able to know how to choose the other user type if he accidentally selects to log in.

Olivia Hartman is an Encost employee. She was told to use the Encost system but was not sure if her password had a capital A or not. She needs to be able to retry logging in if she fails the first time.

- ESGP Feature Options

Liam Bishop is a community user who is playing around with this cool new app he found. He wants to see what features he can access. He needs to be able to see only one feature, the exit option, and graph visualisation, and be clear about how he can use it.

William Prescott is an Encost employee who is bored and wants to fiddle around with the new software they got built. He wants to see what features are available when he logs in with his credentials. He needs to be able to see three features, know what they do, and be able to choose them.

- Loading the Encost Smart Homes Dataset

Noah Donovan is a community user with a slow, computer. He wants to know when the loading of the dataset is taking 1s, and doesn't want to wait a long time.

Encost has a dataset in a text file, they want to be able to load the devices from the file into the system.

Nikayla Molan has an ultra-fast gaming rig and is an Encost Employee. She is in a rush and wants to grab some summary statistics. She wants the data to load quickly and efficiently.

- Categorising Smart Home Device

Encost wants all device information from datasets to be made into objects, which store all device information so that device information associated with a specific device is all in one place for calculating device statistics.

Encost wants their devices to be categorised into device groups from their dataset, which only gives the device's type and name. The categorisations are needed for calculations and visualisations of a device.

- Building a Graph Data Type

Encost wants the Device objects of the system to be stored in a graph data type so that the device information can be calculated for statistics, or easily converted to a graph object.

- Graph Visualisation

Sophia Sinclair is a community member who is not tech-savvy and wants the graph feature to be intuitive and easy to see and understand.

Henry Donovan works at Encost and has a very slow computer. He wants to look at a graph of the different households and devices, without it taking too long, and be notified when it is taking a bit of time.

Charlotte Eastwood is a community user who is included in the Encost dataset. She wants to see the graph and see her household is represented. She needs the graph to differentiate the device categories and make it clear the relationships between devices

- Implementation quality

SoftFlux is responsible for maintaining the software, and would like the code to be high-quality, and commented such that maintainers can understand the code and easily modify it.

4 Sprint Details

Each of the following 3 sprints will cover a period of 4 days.

4.1 Sprint #1 <May 10 - May 13>

4.1.1 Product Backlog Items

- Emma Greenfield is an employee at Encost. She wants to access the extra feature of the system and knows login details. She needs to be able to select to log in.
- Olivia Hartman is a community user, she just wants to use the system without wondering whether she has a login or not. She needs to be able to select to be a community user, without having to think too hard.
- Mia Carmichael wasn't sure if she has the right credentials. She just wants to try if she has the right credentials, but otherwise accesses the system with restricted features. She needs the system to allow her to continue as a community user if she fails.
- Benjamin Mercer is a community user with a very old keyboard. He often mistypes keys, which leads him to choose the wrong thing. He needs to be able to know how to choose the other user type if he accidentally selects to log in.
- Olivia Hartman is an Encost employee. She was told to use the Encost system but was not sure if her password had a capital in it or not. She needs to be able to retry logging in if she fails the first time.
- Liam Bishop is a community user who is playing around with this cool new app he found. He wants to see what features he can access. He needs to be able to see only one feature, graph visualisation, and be clear about how he can use it.
- William Prescott is an Encost employee who is bored and wants to fiddle around with the new software they got built. He wants to see what features are available when he logs in with his credentials. He needs to be able to see three features, know what they do, and be able to choose them.

4.1.2 Sprint Tasks

- (Development) Create Console App structure
- (Development) Create a welcome prompt.

- (Development) Implement the main method
- (Development) Implement 'getUserVersion'
- (Development) Implement 'ApplicationState' class
- (Development) Implement 'DeviceType' enum
- (Development) Create UserVerifier Class structure/constructor
- (Development) Prompt for user credentials
- (Development) Use secure readPassword method built into java
- (Design/Requirements) Add defined User credentials into the program.
- (Development) Verify user credentials and update ApplicationState
- (Development) Print correct login success prompts
- (Development) Offer to 'Continue as community user'/'Try again' after failing to enter the password
- (Requirements/Design) Decide on the ESGP Feature Options prompt design
- (Development) Implement 'selectFeature' in an infinite loop
- (Requirements/Design) Decide how to determine the option selected
- (Development) Implement 'getOptions'

4.1.3 Software Design

4.1.4 Account Login logic

It was not specified where the logic for the account login should go. It was assumed that the login logic would be located in a function called 'accountLogin()' for maintainability and segmentation of logic.

4.1.5 isVerified boolean

The STP has modified the design to use an ApplicationState class to hold the user type, this makes the isVerified boolean redundant as an attribute of the ConsoleApp class. It will be assumed that this will no longer be used.

4.1.6 User input accessibility

A modification has been used to add a string ':' before a user's input, to make it clear to the user the program is expecting input. The STP has used this in its UI specifications, however, does not have it on the same line as the input. It will be held in a constant variable to allow easy modifiability if in future the developer wishes to modify this string.

4.1.7 UserVerifier static

Because the UserVerifier object no longer has to store the 'username' from 'verifyUsername', because of the STP modifications. The UserVerifier class will be implemented as static, as it does not need to be initialised.

4.1.8 Read Password character masking

It was found that the STP incorrectly assumed the functionality of Java's built-in password reading functionality. Instead of masking the password with an '*' the password will just bypass being displayed in the console. It will be implemented so that no '*' will display when entering the password.

4.1.9 Feature options output prompt

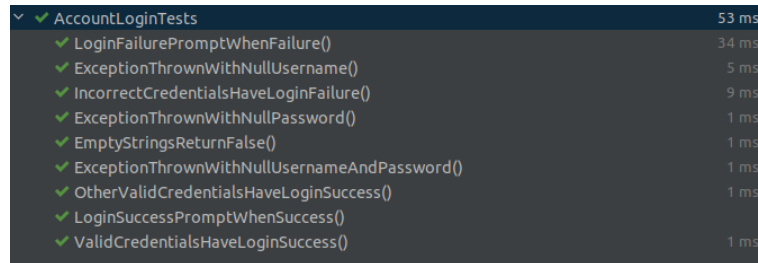
The STP proposes a UI prompt. The prompt is misleading stating 'A dataset has been loaded! What do you want to do?' when the SDS specifies that the encost dataset is loaded and parsed when the user selects one of the options. Instead, the string will be implemented to remove this reference unless the user has loaded a 'custom' dataset - Otherwise, it will just prompt 'What do you want to do?'.

4.1.10 Feature identification

To promote future upgradability, and a more robust method of determining the feature across user types the system will use the array returned by getOptions to determine the switch case to call. This method will allow regardless of the user, to call functions based on what the getOptions function handles and prevents duplication of user type checks. A string constant will be created for each feature, which is printed to the user and uniquely identifies the feature.

4.1.11 Software Testing

Figures 4.1, 4.2, 4.3 demonstrate the 3 features worked on. Options did not pass as the tests were implemented counter to how the program was designed in 4.1.10 - the numbering of the 'getOptions' results has been moved to the selectFeature function, which handles displaying and selecting the options.



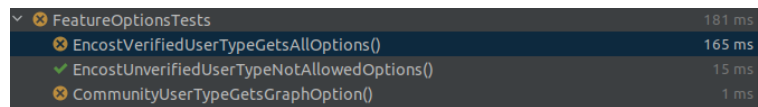
✓ AccountLoginTests	53 ms
✓ LoginFailurePromptWhenFailure()	34 ms
✓ ExceptionThrownWithNullUsername()	5 ms
✓ IncorrectCredentialsHaveLoginFailure()	9 ms
✓ ExceptionThrownWithNullPassword()	1 ms
✓ EmptyStringsReturnFalse()	1 ms
✓ ExceptionThrownWithNullUsernameAndPassword()	1 ms
✓ OtherValidCredentialsHaveLoginSuccess()	1 ms
✓ LoginSuccessPromptWhenSuccess()	
✓ ValidCredentialsHaveLoginSuccess()	1 ms

Figure 4.1: Login tests passing



✓ CategorisingUsersTests	32 ms
✓ UserTypeSetCorrectly()	32 ms

Figure 4.2: UserType setting test passing



✗ FeatureOptionsTests	181 ms
✗ EncostVerifiedUserTypeGetsAllOptions()	165 ms
✓ EncostUnverifiedUserTypeNotAllowedOptions()	15 ms
✗ CommunityUserTypeGetsGraphOption()	1 ms

Figure 4.3: getOptions tests failing

4.1.12 Sprint Task Completion

- Completed - (Development) Create Console App structure
- Completed - (Development) Create a welcome prompt.
- Completed - (Development) Implement the main method
- Completed - (Development) Implement 'getUserVersion'
- Completed - (Development) Implement 'ApplicationState' class
- Completed - (Development) Implement 'DeviceType' enum
- Completed - (Development) Create UserVerifier Class structure/constructor
- Completed - (Development) Prompt for user credentials

- Completed - (Development) Use secure readPassword method built into java
- Completed - (Design/Requirements) Add defined User credentials into the program.
- Completed - (Development) Verify user credentials and update ApplicationState
- Completed - (Development) Print correct login success prompts
- Completed - (Development) Offer to 'Continue as community user'/'Try again' after failing to enter the password
- Completed - (Requirements/Design) Decide on the ESGP Feature Options prompt design
- Completed - (Requirements/Design) Decide how to determine the option selected
- In progress - (Development) Implement getOptions
- Not started - (Development, Requirements) Implement 'selectFeature' in an infinite loop with calls

4.2 Sprint #2 <May 14 - May 17>

4.2.1 Product Backlog Items

- Noah Donovan is a community user with a slow, computer. He wants to know when the loading of the dataset is taking 41s, and doesn't want to wait a long time.
- Encost has a dataset in a text file, they want to be able to load the devices from the file into the system.
- Nikayla Molan has an ultra-fast gaming rig and is an Encost Employee. She is in a rush and wants to grab some summary statistics. She wants the data to load quickly and efficiently.
- Encost wants all device information from datasets to be made into objects, which store all device information so that device information associated with a specific device is all in one place for calculating device statistics.
- Encost wants their devices to be categorised into device groups from their dataset, which only gives the device's type and name. The categorisations are needed for calculations and visualisations of a device.
- Encost wants the Device objects of the system to be stored in a graph data type so that the device information can be calculated for statistics, or easily converted to a graph object.
- Sophia Sinclair is someone from the community who has installed this program on her Windows computer. She isn't particularly tech-savvy and often has to squint at her screen. She needs to be able to see a nice, clear and simple graph which is easy for her to understand.
- Henry Donovan works at Encost and has a very slow computer. He wants to look at a graph of the different households and devices. He needs to be notified if the process is taking too long, that it is still processing and his computer has not just frozen.
- Charlotte Eastwood is a community user who is included in the Encost dataset. She wants to see the graph and find how her household is represented, she knows the device categories of all her devices. She needs the graph to differentiate the device categories and make it clear the relationships between devices

4.2.2 Sprint Tasks

- (Carry Over)(Development) Implement `getOptions`
- (Carry Over)(Development, Requirements) Implement `'selectFeature'` in an infinite loop with calls

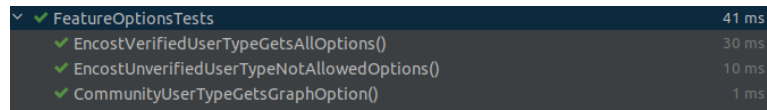
- (Development) Implement the FileParser
- (Development) Implement the Device class to hold device information
- (Development) Create DeviceType Enum structure/constructor
- (Development) Trigger loading of the dataset on Graph Visualisation or summary statistics
- (Development) Implement the loadFile function in the ConsoleApp
- (Development) Inform the user when loading the dataset takes a long to load
- (Development) Implement DeviceCategory
- (Development) Implement DeviceCategoriser
- (Development) Implement adjacency matrix
- (Development) Create GraphVisualiser structure/constructor
- (Development) Implement GraphVisualiser convertGraph
- (Development) Implement GraphVisualiser visualise
- (Requirements/Design) Decide on Simple and Fast GraphStream UI design, based on SDS
- (Development) Add styling to Graph, based on category
- (Development, Requirements) Inform the user when visualising the graph takes a long time to load
- (Development) Add the encost dataset to the program

4.2.3 Software Design

- Created function 'determineDeviceType' to convert the string to a DeviceType enum
- Using limitations of GraphStream 1.3, increase size of nodes, and have Device name as the label as per the SDS
- Used a Boolean array, instead of an Integer array for the adjacency matrix. The integer matrix would only toggle between 1 and 0, so could be simplified to a boolean array
- Created a function called 'checkTiming' to handle checking non-function length taken - Every 1 second the user is notified, and the dataset is deemed too large if it takes 10 seconds.

- DeviceCategoriser had its function set to static, as there is no need to instantiate something which only uses the parameters of the function (which is the only function in the class).

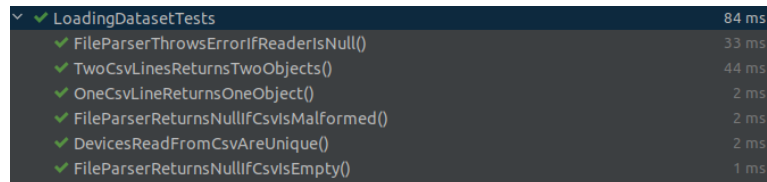
4.2.4 Software Testing



A screenshot of a test runner interface showing a list of tests under the 'FeatureOptionsTests' category. Each test is preceded by a green checkmark, indicating it passed. The tests and their durations are: EncostVerifiedUserTypeGetsAllOptions() (30 ms), EncostUnverifiedUserTypeNotAllowedOptions() (10 ms), and CommunityUserTypeGetsGraphOption() (1 ms).

✓ FeatureOptionsTests	41 ms
✓ EncostVerifiedUserTypeGetsAllOptions()	30 ms
✓ EncostUnverifiedUserTypeNotAllowedOptions()	10 ms
✓ CommunityUserTypeGetsGraphOption()	1 ms

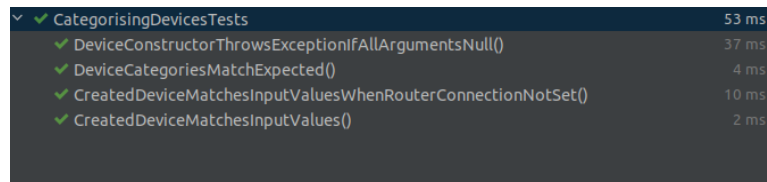
Figure 4.4: getOptions tests passing



A screenshot of a test runner interface showing a list of tests under the 'LoadingDatasetTests' category. Each test is preceded by a green checkmark, indicating it passed. The tests and their durations are: FileParserThrowsErrorIfReaderIsNull() (33 ms), TwoCsvLinesReturnsTwoObjects() (44 ms), OneCsvLineReturnsOneObject() (2 ms), FileParserReturnsNullIfCsvIsMalformed() (2 ms), DevicesReadFromCsvAreUnique() (2 ms), and FileParserReturnsNullIfCsvIsEmpty() (1 ms).

✓ LoadingDatasetTests	84 ms
✓ FileParserThrowsErrorIfReaderIsNull()	33 ms
✓ TwoCsvLinesReturnsTwoObjects()	44 ms
✓ OneCsvLineReturnsOneObject()	2 ms
✓ FileParserReturnsNullIfCsvIsMalformed()	2 ms
✓ DevicesReadFromCsvAreUnique()	2 ms
✓ FileParserReturnsNullIfCsvIsEmpty()	1 ms

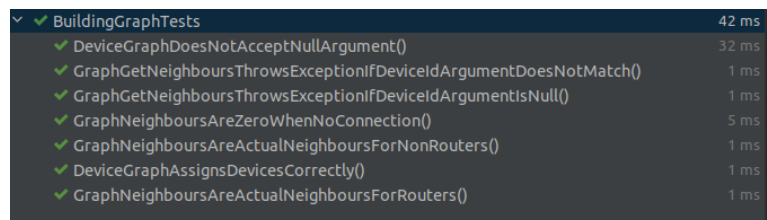
Figure 4.5: FileParser tests pass



A screenshot of a test runner interface showing a list of tests under the 'CategorisingDevicesTests' category. Each test is preceded by a green checkmark, indicating it passed. The tests and their durations are: DeviceConstructorThrowsExceptionIfAllArgumentsNull() (37 ms), DeviceCategoriesMatchExpected() (4 ms), CreatedDeviceMatchesInputValuesWhenRouterConnectionNotSet() (10 ms), and CreatedDeviceMatchesInputValues() (2 ms).

✓ CategorisingDevicesTests	53 ms
✓ DeviceConstructorThrowsExceptionIfAllArgumentsNull()	37 ms
✓ DeviceCategoriesMatchExpected()	4 ms
✓ CreatedDeviceMatchesInputValuesWhenRouterConnectionNotSet()	10 ms
✓ CreatedDeviceMatchesInputValues()	2 ms

Figure 4.6: Device Categorisation Tests



A screenshot of a test runner interface showing a list of tests under the 'BuildingGraphTests' category. Each test is preceded by a green checkmark, indicating it passed. The tests and their durations are: DeviceGraphDoesNotAcceptNullArgument() (32 ms), GraphGetNeighboursThrowsExceptionIfDeviceIdArgumentDoesNotMatch() (1 ms), GraphGetNeighboursThrowsExceptionIfDeviceIdArgumentIsNull() (1 ms), GraphNeighboursAreZeroWhenNoConnection() (5 ms), GraphNeighboursAreActualNeighboursForNonRouters() (1 ms), DeviceGraphAssignsDevicesCorrectly() (1 ms), and GraphNeighboursAreActualNeighboursForRouters() (1 ms).

✓ BuildingGraphTests	42 ms
✓ DeviceGraphDoesNotAcceptNullArgument()	32 ms
✓ GraphGetNeighboursThrowsExceptionIfDeviceIdArgumentDoesNotMatch()	1 ms
✓ GraphGetNeighboursThrowsExceptionIfDeviceIdArgumentIsNull()	1 ms
✓ GraphNeighboursAreZeroWhenNoConnection()	5 ms
✓ GraphNeighboursAreActualNeighboursForNonRouters()	1 ms
✓ DeviceGraphAssignsDevicesCorrectly()	1 ms
✓ GraphNeighboursAreActualNeighboursForRouters()	1 ms

Figure 4.7: Build Graph tests passing

4.2.5 Sprint Task Completion

- (Carry Over)(Development) Implement getOptions
- (Carry Over)(Development, Requirements) Implement 'selectFeature' in an infinite loop with calls
- Completed - (Development) Implement the FileParser

- Completed - (Development) Implement the Device class to hold device information
- Completed - (Development) Create DeviceType Enum structure/constructor
- Completed - (Development) Trigger loading of the dataset on Graph Visualisation or summary statistics
- Completed - (Development) Implement the loadFile function in the ConsoleApp
- Completed - (Development) Inform the user when loading the dataset takes a long to load
- Completed - (Development) Implement DeviceCategory
- Completed - (Development) Implement DeviceCategoriser
- Completed - (Development) Implement adjacency matrix
- Completed - (Development) Create GraphVisualiser structure/constructor
- Completed - (Development) Implement GraphVisualiser convertGraph
- Completed - (Development) Implement GraphVisualiser visualise
- Completed - (Requirements/Design) Decide on Simple and Fast GraphStream UI design, based on SDS
- Completed - (Development, Requirements) Inform the user when visualising the graph takes a long time to load
- In Progress, will be rolled over - (Development) Add styling to Graph, based on category
- In Progress, will be rolled over - (Development) Add the encost dataset to the program

4.3 Sprint #3 <May 18 - May 21>

4.3.1 Product Backlog Items

- Charlotte Eastwood is a community user who is included in the Encost dataset. She wants to see the graph and see her household is represented. She needs the graph to differentiate the device categories and make it clear the relationships between devices
- Encost has a dataset in a text file, they want to be able to load the devices from the file into the system.
- SoftFlux is responsible for maintaining the software, and would like the code to be high-quality, and commented such that maintainers can understand the code and easily modify it.

4.3.2 Sprint Tasks

- (Carry Over)(Development) Add styling to Graph, based on category
- (Development) Add a Colour key to the console to indicate what categories are what in the graph.
- (Carry Over)(Development, Requirements) Add the encost dataset to the program
- (Development, Bug fix) Handle non-existent encost dataset file
- (Bug fix) Fix broken renaming
- (Development) Renaming, cleaning, and commenting
- (Development, Design) Comment and format code

4.3.3 Software Design

Figure 4.8 demonstrates how the sprints and user stories were implemented over the 12 days of development.

Design Decisions:

- loginSuccessPrompt function was removed, as it only returned 1 of 2 strings, which are each called in different branches - and extracting away provides less clarity to surrounding code.
- Used a stylesheet, loaded and converted to a string to prevent referencing an absolute path for the GraphStream library.
- Added label for 'Device Name' to fit with SDS graph diagram of the design - GraphStream 1.3 does not support changing the label to be over the circle like in the SDS.

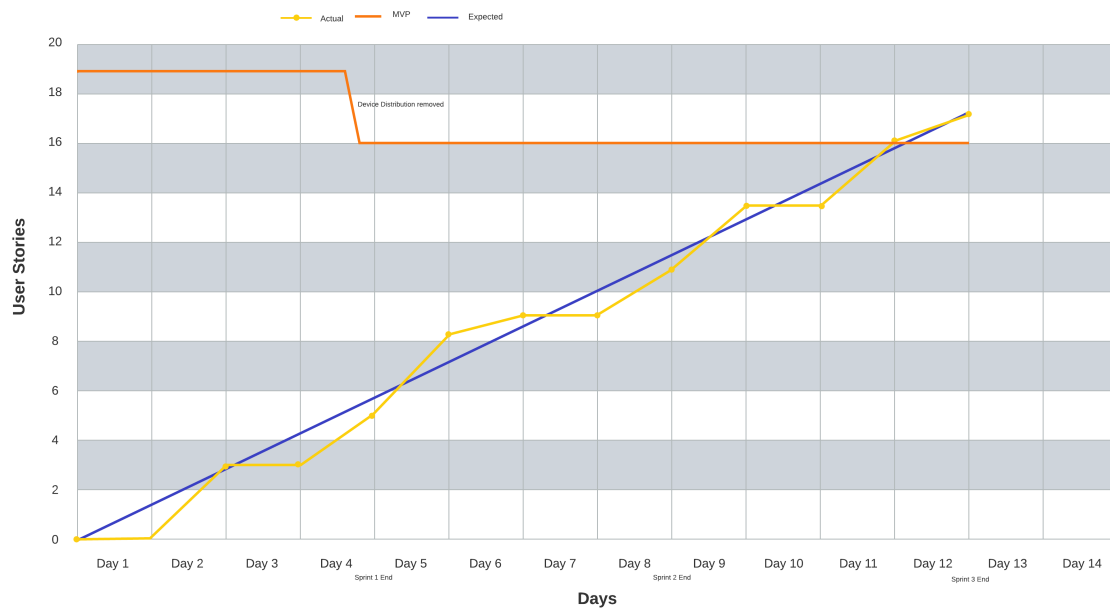
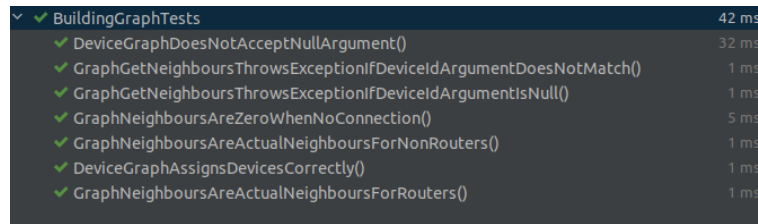


Figure 4.8: Burn up chart

- When converting CSV DeviceType to enum, decided to convert the input to lowercase and compare that way to avoid capitalisation differences.
- Printed the colour key to the console, as GraphStream does not support an easy colour key.

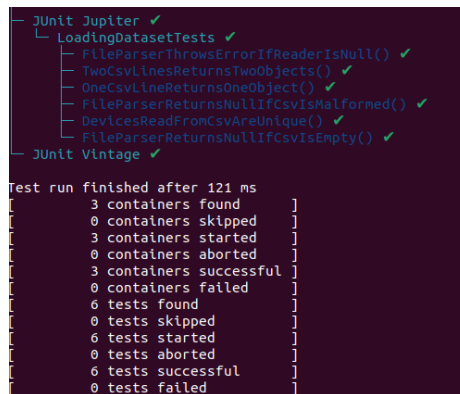
4.3.4 Software Testing



A screenshot of a test runner interface showing a list of tests under the category 'BuildingGraphTests'. Each test is preceded by a green checkmark, indicating it passed. The tests and their durations are as follows:

Test Name	Duration
DeviceGraphDoesNotAcceptNullArgument()	32 ms
GraphGetNeighboursThrowsExceptionIfDeviceIdArgumentDoesNotMatch()	1 ms
GraphGetNeighboursThrowsExceptionIfDeviceIdArgumentIsNull()	1 ms
GraphNeighboursAreZeroWhenNoConnection()	5 ms
GraphNeighboursAreActualNeighboursForNonRouters()	1 ms
DeviceGraphAssignsDevicesCorrectly()	1 ms
GraphNeighboursAreActualNeighboursForRouters()	1 ms

Figure 4.9: Testing graph displays



A screenshot of a terminal window showing JUnit test results. The tests are grouped under 'JUnit Jupiter' and 'JUnit Vintage'. All tests passed, indicated by green checkmarks. Below the test results, a summary of the test run is provided:

```
Test run finished after 121 ms
  3 containers found
  0 containers skipped
  3 containers started
  0 containers aborted
  3 containers successful
  0 containers failed
  6 tests found
  0 tests skipped
  6 tests started
  0 tests aborted
  6 tests successful
  0 tests failed
```

Figure 4.10: Loading dataset testing

4.3.5 Sprint Task Completion

- Completed - (Carry Over)(Development) Add styling to Graph, based on category
- Completed - (Development) Add a Colour key to the console to indicate what categories are what in the graph.
- Completed - (Carry Over)(Development, Requirements) Add the encost dataset to the program
- Completed - (Development, Bug fix) Handle non-existent encost dataset file
- Completed - (Bug fix) Fix broken renaming
- Completed - (Development) Renaming, cleaning, and commenting
- Completed - (Development, Design) Comment and format code

5 Conclusion

The developed Encost Smart Graph Project is a software that allows Encost to allow their community and employees to visualise smart home data in a simple way. It was developed with two user type Community user, or an Encost user, which the different userbase can access.

By logging in using one of 10 provided accounts, coded into the system for security, an Encost employee can access 2 other (unimplemented) features of 'summary statistics', and 'load a custom dataset'. The graph visualisation feature, available to both Community users and Encost users loads an 'Encost Smart Homes Dataset.txt' file, and loads all of the individual devices into objects which are then mapped to one another in a graph data type.

When the user uses a feature that requires a dataset to be loaded, only then will the file be parsed. If the loading of the Graph Visualisation, or parsing the file, takes longer than 1s the user will be notified.

The system provides a simple graph visualisation using colours to differentiate devices by their category, for which the colour key is printed in the console. All loading and displaying runs fast and use simple and easy-to-understand prompts. Any errors are printed to the console in an easily readable format so that even non-tech-savvy people can make good use of this software.